AI-Based Network Intrusion Detection System

Final-Year Project Reference Document — Extended Edition

Prepared with Claude

August 2026

1. Project Overview
2. Why This Project Matters
3. Problem Statement
4. Main Objective
5. NSL-KDD Dataset
6. Alternative and Complementary Datasets
    - 6.1 CICIDS2017 (Canadian Institute for Cybersecurity)
    - 6.2 UNSW-NB15 (Australian Centre for Cyber Security)
    - 6.3 KDD Cup 1999 (original, for historical comparison only)
    - 6.4 NSL-KDD download sources
7. Proposed System Architecture
8. Machine Learning Models
    - 8.1 Random Forest
    - 8.2 Decision Tree
    - 8.3 Support Vector Machine (SVM)
    - 8.4 XGBoost
    - 8.5 Logistic Regression / Naive Bayes (optional extras)
9. Deep Learning
10. Binary vs. Multiclass Classification
11. Data Preprocessing
12. Model Evaluation
13. Proposed Dashboard
14. Recommended Technology Stack
15. GitHub Project References (25+)
    - 15.1 Direct NSL-KDD classification projects (ML-focused)
    - 15.2 Deep learning-focused projects
    - 15.3 Multi-dataset / broader-scope projects
    - 15.4 Dashboard / deployment-focused projects
    - 15.5 Dataset source / mirror repositories
    - 15.6 GitHub Topic pages (use these to discover newer projects continuously)
    - 15.7 Useful GitHub searches (always up to date)
16. Which Projects Should You Study First?
17. Research Questions for Your Proposal
18. Expected Final Result
19. Ethical, Legal, and Practical Considerations
20. Suggested Report / Chapter Structure
21. Final Recommendation

1. Project Overview

An **AI-Based Network Intrusion Detection System (IDS)** is a cybersecurity system that monitors network traffic and identifies whether that traffic is **normal or potentially malicious**.

Traditional intrusion detection systems often depend on predefined attack signatures or manually written rules. These systems are effective against known attacks but struggle with novel or slightly modified attacks, because a signature that does not exactly match a rule will simply pass through undetected. An **AI-based IDS** instead uses **Machine Learning (ML)** and **Deep Learning (DL)** algorithms to learn statistical patterns from network traffic and generalize to traffic it has not seen before.

This document uses the **NSL-KDD dataset** as the primary benchmark, since it is the most widely used dataset in academic IDS research and is small enough to be trained on a laptop, but the document also introduces two modern alternative/complementary datasets (**CICIDS2017** and **UNSW-NB15**) so the project can be extended beyond a single benchmark if a stronger final-year submission is desired.

Basic workflow

```
Network Traffic / NSL-KDD
          |
   Data Preprocessing
          |
   Feature Engineering
          |
     ML / DL Models
          |
    Attack Detection
          |
 Normal / Attack Type
          |
   Alert & Dashboard
```

---

2. Why This Project Matters

Network intrusion detection sits at the intersection of three trends that make it a strong final-year topic:

1. **Growing attack surface.** More devices (IoT, cloud workloads, remote endpoints) means more traffic to monitor and more entry points for attackers.
2. **Signature fatigue.** Purely rule-based systems (e.g., classic Snort rules) require constant manual updates and miss zero-day or polymorphic attacks.
3. **Maturity of ML tooling.** Libraries like scikit-learn, XGBoost, and TensorFlow/PyTorch make it realistic for a student to build, train, and evaluate multiple models within a single semester.

A well-scoped project does not need to invent a new algorithm — it needs to **apply existing algorithms rigorously, evaluate them fairly, and present the results clearly**, which is exactly what examiners look for in a final-year project.

---

3. Problem Statement

Cyberattacks such as denial-of-service (DoS) attacks, network scanning/reconnaissance, unauthorized remote access, and privilege escalation can cause serious damage to computer networks, ranging from service outages to data breaches.

Traditional signature-based IDS solutions may have difficulty identifying previously unseen or modified attack patterns, since they rely on matching traffic against a fixed rule database.

Therefore, this project proposes an **AI-based intrusion detection system** that learns network-traffic patterns using machine-learning and deep-learning algorithms and automatically classifies suspicious connections, with the goal of generalizing to attack variants that were not explicitly programmed into a rule set.

---

4. Main Objective

> ***To design and implement an AI-based network intrusion detection system capable of identifying and classifying suspicious network traffic using machine-learning and deep-learning techniques.***

Specific objectives

1. Study existing network intrusion detection techniques (signature-based, anomaly-based, hybrid).
2. Use the NSL-KDD dataset (and optionally CICIDS2017/UNSW-NB15) for model development.
3. Preprocess and transform raw network-traffic data into model-ready features.
4. Train multiple machine-learning classifiers (Decision Tree, Random Forest, SVM, XGBoost).
5. Implement at least one deep-learning model (ANN, CNN, or LSTM).
6. Compare the performance of ML and DL models on the same train/test split.
7. Evaluate models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices.
8. Develop a simple dashboard for intrusion prediction and alerting.
9. Identify the network features that most influence detection performance (feature importance / SHAP).
10. Discuss the practical limitations of deploying the model on live traffic.

---

5. NSL-KDD Dataset

The **NSL-KDD dataset** is a refined version of the older **KDD Cup 1999** dataset, published by the Canadian Institute for Cybersecurity (University of New Brunswick) to remove redundant and duplicate records that biased earlier research toward frequent record types.

Main categories

Category	Meaning
Normal	Legitimate network traffic
DoS	Denial-of-Service attacks

Probe	Network scanning/reconnaissance
R2L	Remote-to-Local attacks
U2R	User-to-Root attacks

Feature families (41 features total)

- **Basic features** — protocol type, service, connection duration, flag, source/destination bytes.
- **Content features** — number of failed logins, whether root shell was obtained, number of file creations.
- **Time-based traffic features** — connection counts and error rates within a 2-second window.
- **Host-based traffic features** — connection counts and error rates aggregated per destination host.

Why NSL-KDD over raw KDD'99

- No redundant records in the training set, so classifiers are not biased toward the most frequent connection types.
- No duplicate records in the test set, so scores are not inflated by methods that simply memorize frequent patterns.
- A manageable size (~148,000 records) that trains quickly, even on a laptop CPU.

Known limitation

NSL-KDD reflects network conditions from the late 1990s. It does not contain modern attack types (e.g., ransomware traffic patterns, encrypted-channel exfiltration, IoT botnet floods). For this reason, Section 6 introduces two newer datasets that can be used to strengthen a final-year project or included as a "generalization" experiment.

---

6. Alternative and Complementary Datasets

Including one of these as a secondary dataset — even just for a binary classification comparison — makes the project noticeably stronger than a single-dataset submission, because it demonstrates that a model generalizes rather than overfitting to one benchmark's quirks.

6.1 CICIDS2017 (Canadian Institute for Cybersecurity)

A modern labeled dataset covering benign traffic and common modern attacks: Brute Force, Heartbleed, Botnet, DoS, DDoS, Web Attacks, and Infiltration, captured over five days of realistic traffic.

- Official page: https://www.unb.ca/cic/datasets/ids-2017.html
- Kaggle mirror: https://www.kaggle.com/datasets/cicdataset/cicids2017

6.2 UNSW-NB15 (Australian Centre for Cyber Security)

Contains nine modern attack categories (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms) generated with the IXIA PerfectStorm tool.

- Official page: https://research.unsw.edu.au/projects/unsw-nb15-dataset
- Kaggle mirror: https://www.kaggle.com/datasets/mrwellsdavid/unsw-nb15

6.3 KDD Cup 1999 (original, for historical comparison only)

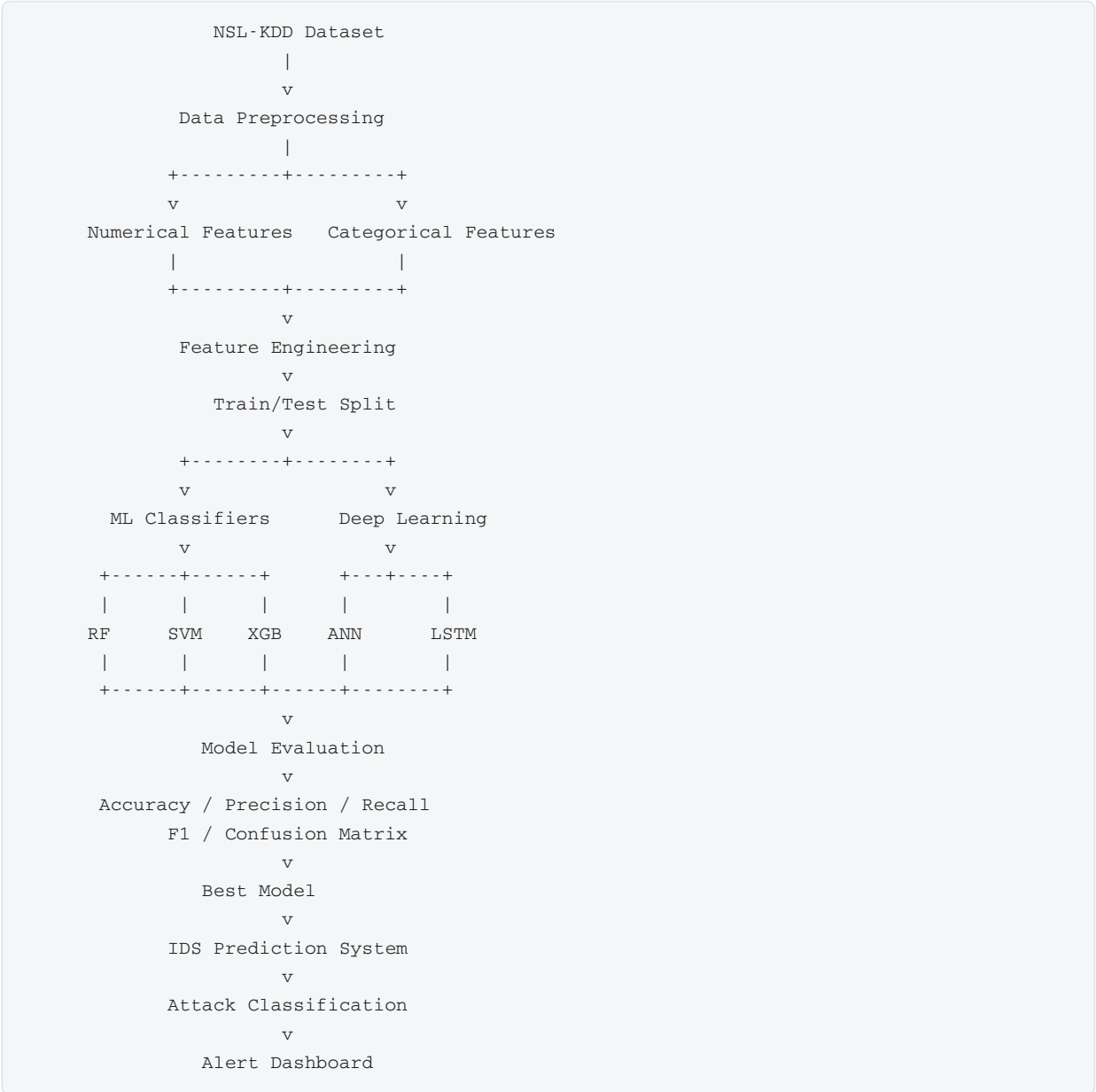
- Official page: http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html

6.4 NSL-KDD download sources

- Official CIC page: https://www.unb.ca/cic/datasets/nsl.html
- Mirror with raw ARFF/TXT files: https://github.com/jmnwong/NSL-KDD-Dataset
- Kaggle notebook + dataset mirror: https://www.kaggle.com/code/eneskosar19/intrusion-detection-system-nsl-kdd
- Kaggle dataset (CSV format, easy to load with pandas): https://www.kaggle.com/datasets/hassan06/nslkdd

---

7. Proposed System Architecture



---

8. Machine Learning Models

8.1 Random Forest

Random Forest uses an ensemble of decision trees, each trained on a bootstrapped sample with a random subset of features, and averages their votes. It is a strong choice for NSL-KDD because the data is primarily structured/tabular, and it naturally provides **feature-importance scores**, which are useful for explaining predictions to non-technical stakeholders or examiners.

8.2 Decision Tree

A single decision tree is a simple, interpretable baseline. You can visualize the tree and explain, in plain language, why a particular connection was classified as suspicious — useful for a project demo.

8.3 Support Vector Machine (SVM)

SVM finds the boundary (hyperplane) that best separates traffic classes, maximizing the margin between them. It is sensitive to feature scaling, so it provides a good stress-test for your preprocessing pipeline and a useful comparison point against tree-based models.

8.4 XGBoost

XGBoost is a gradient-boosting algorithm that builds trees sequentially, each correcting the errors of the previous one. It typically achieves the best raw accuracy among classical ML models on tabular intrusion-detection data and is worth including as your “strong baseline.”

8.5 Logistic Regression / Naive Bayes (optional extras)

Useful as fast, simple baselines to show that your more complex models genuinely add value rather than just adding complexity.

---

9. Deep Learning

Include at least one deep-learning model; you do **not** need to implement every architecture below. For a final-year project, one well-designed DL experiment, properly evaluated, is preferable to several poorly evaluated models.

9.1 Artificial Neural Network (ANN / MLP)

```
Network Features
       |
 Input Layer
       |
 Hidden Layer
       |
 Hidden Layer
```

```
       |
 Output Layer
       |
Attack Classification
```

9.2 1D-CNN

Treats the fixed-length feature vector as a 1D "signal" and applies convolutional filters to detect local patterns among adjacent features. Several of the reference repositories below (Section 15) report CNNs outperforming plain ANNs by a few percentage points on NSL-KDD.

9.3 LSTM

LSTM can be investigated when network records are represented in a meaningful sequential form (e.g., a sequence of connections from the same host over time, rather than one isolated record).

> *Research question: Can sequential modeling improve intrusion detection compared with conventional machine-learning classifiers that treat each connection independently?*

9.4 Autoencoder (unsupervised anomaly detection)

```
Normal Traffic
      |
   Encoder
      |
Latent Representation
      |
   Decoder
      |
Reconstructed Traffic
      |
Reconstruction Error
      |
Possible Anomaly
```

An autoencoder is trained **only on normal traffic**. At inference time, traffic that the model reconstructs poorly (high reconstruction error) is flagged as anomalous. This is valuable because it can, in principle, catch attack types the model was never explicitly trained on — a genuine "zero-day" detection angle that examiners tend to find impressive.

---

10. Binary vs. Multiclass Classification

Experiment 1 — Binary Classification

```
Traffic
   |
Normal OR Attack
```

Simpler, and useful as a sanity-check baseline before moving to the harder problem.

Experiment 2 — Multiclass Classification

```
Traffic
   |
 +------+-----+-------+-----+-----+
Normal  DoS  Probe    R2L   U2R
```

Multiclass classification provides a more detailed analysis and is generally more interesting for a final-year project, because it forces you to confront class imbalance (R2L and U2R have very few samples) and per-class evaluation.

---

11. Data Preprocessing

- **Categorical encoding** — convert fields such as protocol type (TCP/UDP/ICMP), service, and flag into numeric representations (one-hot encoding or label encoding).
- **Feature scaling** — particularly important for SVM and neural networks; use `StandardScaler` or `MinMaxScaler`.
- **Missing/invalid values** — identify and handle missing or malformed records before training.
- **Feature selection** — remove irrelevant or redundant features where justified (correlation analysis, mutual information, or feature-importance pruning).
- **Class imbalance** — some attack categories (especially U2R and R2L) are severely under-represented. Techniques worth trying:
  - SMOTE / ADASYN oversampling of minority classes.
  - Class-weighted loss functions.
  - Reporting **per-class** precision, recall, and F1-score rather than relying only on overall accuracy, which can be misleadingly high on imbalanced data.

---

12. Model Evaluation

Metric	Purpose
Accuracy	Overall prediction correctness
Precision	Reliability of attack predictions
Recall	Ability to detect actual attacks
F1-score	Balance between precision and recall
ROC-AUC	Threshold-independent discrimination
Confusion Matrix	Shows class-by-class errors
Training Time	Practical model comparison
Prediction Time	Potential deployment performance

For cybersecurity, **recall is especially important**, because a model that misses many actual attacks (false negatives) may be dangerous to rely on even if its overall accuracy looks high. A useful discussion point in your report is the trade-off between recall (catching attacks) and precision (avoiding alert fatigue from false positives).

---

13. Proposed Dashboard

```
+-----------------------------------------+
|       AI NETWORK INTRUSION DETECTION    |
+-----------------------------------------+
|                                         |
| Total Connections:       12,453         |
| Normal Traffic:          10,891         |
| Suspicious Traffic:       1,562         |
|                                         |
| Latest Detection:                      |
|                                         |
| Attack Type:             DoS            |
| Confidence:              96.4%          |
| Severity:                HIGH           |
|                                         |
| [Traffic] [Alerts] [Model Results]     |
+-----------------------------------------+
```

Streamlit is a convenient choice for a student project because you can build a functional interface using only Python, without a separate frontend framework. For a slightly more production-flavored deliverable, several reference projects in Section 15 instead expose a **FastAPI** prediction endpoint behind a Streamlit or Flask front end, and containerize the whole thing with Docker — worth doing if you want a "deployment" chapter in your report.

---

14. Recommended Technology Stack

Component	Technology
Programming	Python
Dataset	NSL-KDD (+ optionally CICIDS2017 / UNSW-NB15)
Data Processing	Pandas, NumPy
ML	Scikit-learn
Advanced ML	XGBoost, LightGBM (optional)
Deep Learning	TensorFlow/Keras or PyTorch
Explainability	SHAP
Visualization	Matplotlib / Plotly
Dashboard	Streamlit (or Flask + FastAPI)
Development	Jupyter Notebook + VS Code
Deployment (optional)	Docker
Optional Database	SQLite/PostgreSQL

---

15. GitHub Project References (25+)

The list below groups references so you can navigate quickly. Treat every repository as a **reference implementation to study, not code to submit unchanged**. Check each repository's license before reusing code or assets, and always cite the author in your own report if you adapt any of their approach.

15.1 Direct NSL-KDD classification projects (ML-focused)

1. **AchlysX/NSL-KDD-intrusion-detection** — Decision Tree, Random Forest, SVM with SMOTE balancing. https://github.com/AchlysX/NSL-KDD-intrusion-detection
2. **CynthiaKoopman/Network-Intrusion-Detection** — Classic ML pipeline on NSL-KDD, widely cited student reference. https://github.com/CynthiaKoopman/Network-Intrusion-Detection
3. **Deepthi10/Intrusion-Detection-using-Machine-Learning-on-NSL-KDD-dataset** — Data-mining preprocessing + classifiers. https://github.com/Deepthi10/Intrusion-Detection-using-Machine-Learning-on-NSL--KDD-dataset
4. **Mamcose/NSL-KDD-Network-Intrusion-Detection** — Random Forest, KNN, SVM plus ensemble voting. https://github.com/Mamcose/NSL-KDD-Network-Intrusion-Detection
5. **flemingpatel/nsl_kdd_network_intrusion_detection** — Multiclass classification with EDA + model comparison. https://github.com/flemingpatel/nsl_kdd_network_intrusion_detection
6. **davidreynaldositorus/IntrusionDetectionSystem** — Focused SVM implementation on NSL-KDD. https://github.com/davidreynaldositorus/IntrusionDetectionSystem
7. **thinline72/nsl-kdd** — PySpark + scikit-learn approaches, Dockerized notebook, feature-selection notes. https://github.com/thinline72/nsl-kdd
8. **virenchav10/ids_nslkdd** — Random Forest + Isolation Forest + Autoencoder with SHAP explainability. https://github.com/virenchav10/ids_nslkdd
9. **Tari-elham/NSL-KDD-Intrusion-Detection** — Simple, beginner-friendly binary classification baseline. https://github.com/Tari-elham/NSL-KDD-Intrusion-Detection

15.2 Deep learning-focused projects

10. **Akhil-Joseph11/Intrusion-Detection-using-the-NSL-KDD-dataset** — DNN vs. 1D-CNN comparison, ADASYN for class imbalance, per-class breakdown. https://github.com/Akhil-Joseph11/Intrusion-Detection-using-the-NSL-KDD-dataset
11. **munirKarsli/Network-Intrusion-Detection-With-Deep-Learning-On-Nsl-Kdd-Dataset** — CNN-LSTM hybrid notebooks for binary and multiclass tasks. https://github.com/munirKarsli/Network-Intrusion-Detection-With-Deep-Learning-On-Nsl-Kdd-Dataset
12. **jouleffect/Intrusion-Detection-System** — CNN reproduction of a published NSL-KDD paper, includes citations you can reuse. https://github.com/jouleffect/Intrusion-Detection-System
13. **dharaneishvc/Intrusion-detection-DL-ML** — Side-by-side ML/DL comparison. https://github.com/dharaneishvc/Intrusion-detection-DL-ML
14. **parishree-gupta/Network-Intrusion-Detection-using-MachineLearning** — Combines multiple ML approaches with LSTM, binary and multiclass. https://github.com/parishree-gupta/Network-Intrusion-Detection-using-MachineLearning
15. **vinayakumarr/Network-Intrusion-Detection** — Well-cited research code covering KDDCup'99, NSL-KDD, and UNSW-NB15 with CNN/RNN variants and linked IEEE papers. https://github.com/vinayakumarr/Network-Intrusion-Detection

15.3 Multi-dataset / broader-scope projects

16. **mohammedAcheddad/AI-Based-Network-IDS_ML-DL** — Combines NSL-KDD and UNSW-NB15 for a more generalizable IDS; good model for Section 6's "secondary dataset" idea. https://github.com/mohammedAcheddad/AI-Based-Network-IDS_ML-DL

17. **ranimhassine/Intrusion_Detection_NSL_KDD** — Deep neural network / ensemble approach with a clear README structure worth modeling your own report on. https://github.com/ranimhassine/Intrusion_Detection_NSL_KDD

15.4 Dashboard / deployment-focused projects

18. **deepikasingh007/NIDS** — NSL-KDD + ML + Streamlit dashboard, closest match to Section 13's proposed dashboard. https://github.com/deepikasingh007/NIDS
19. **shenal00/network-intrusion-detection-ml** — Random Forest + FastAPI prediction endpoint + real-time dashboard, AUC reporting. https://github.com/shenal00/network-intrusion-detection-ml
20. **debjotyms/streamlit-ml-intrusion-detection-system** — Clean, minimal Streamlit + Random Forest app; good structural template for a student project. https://github.com/debjotyms/streamlit-ml-intrusion-detection-system
21. **vicky60629/Network-Intrusion-Detection-System** — Flask web app with a manual input form, includes a clear roadmap of "future enhancements" worth borrowing for your own conclusion chapter. https://github.com/vicky60629/Network-Intrusion-Detection-System
22. **flatmarstheory/ai-powered-ids-for-home-networks** — Full pipeline including live packet sniffing, threat-intel API lookups, and a Streamlit dashboard, packaged with Docker. https://github.com/flatmarstheory/ai-powered-ids-for-home-networks

15.5 Dataset source / mirror repositories

23. **jmnwong/NSL-KDD-Dataset** — Raw NSL-KDD files (ARFF and TXT/CSV formats) mirrored on GitHub. https://github.com/jmnwong/NSL-KDD-Dataset

15.6 GitHub Topic pages (use these to discover newer projects continuously)

24. **nsl-kdd topic** — https://github.com/topics/nsl-kdd
25. **nsl-kdd-dataset topic** — https://github.com/topics/nsl-kdd-dataset
26. **network-intrusion-detection topic** — https://github.com/topics/network-intrusion-detection
27. **intrusion-detection-system topic** — https://github.com/topics/intrusion-detection-system
28. **intrusion-detection topic** — https://github.com/topics/intrusion-detection

15.7 Useful GitHub searches (always up to date)

29. NSL-KDD + LSTM — https://github.com/search?q=NSL-KDD+LSTM&type=repositories
30. NSL-KDD + CNN — https://github.com/search?q=NSL-KDD+CNN&type=repositories
31. NSL-KDD + XGBoost — https://github.com/search?q=NSL-KDD+XGBoost&type=repositories
32. NSL-KDD + Autoencoder — https://github.com/search?q=NSL-KDD+autoencoder&type=repositories
33. Network IDS + Streamlit — https://github.com/search?q=streamlit+network+intrusion+detection&type=repositories
34. Anomaly Detection + Network Intrusion — https://github.com/search?q=anomaly+detection+network+intrusion&type=repositories

---

16. Which Projects Should You Study First?

Priority	Repository	Main reason
Highest	vinayakumarr/Network-Intrusion-Detection	Peer-reviewed methodology, three datasets, citable IEEE papers
Highest	Akhil-Joseph11/...NSL-KDD-dataset	Clear DNN vs. CNN comparison with class-imbalance handling
High	mohammedAcheddad/AI-Based-Network-IDS_ML-DL	Multi-dataset generalization angle
High	deepikasingh007/NIDS	Streamlit dashboard template
High	shenal00/network-intrusion-detection-ml	FastAPI + dashboard + AUC reporting, deployment-oriented
Medium	CynthiaKoopman/Network-Intrusion-Detection	Solid, simple ML baseline to start from
Medium	virenchav10/ids_nslkdd	SHAP explainability example
Medium	GitHub topic/search pages	Keep discovering newer repositories as you work

Suggested study order

1. Start with **CynthiaKoopman** or **AchlysX** for a clean ML baseline pipeline.
2. Move to **Akhil-Joseph11** or **munirKarsli** for the deep-learning half of your comparison.
3. Study **vinayakumarr** for how to properly cite and structure a multi-dataset evaluation.
4. Use **deepikasingh007** or **shenal00** as your dashboard/deployment template.
5. Then **build your own implementation** rather than copying any single repository — combine ideas, and make sure your report explains *why* you made each design choice.

---

17. Research Questions for Your Proposal

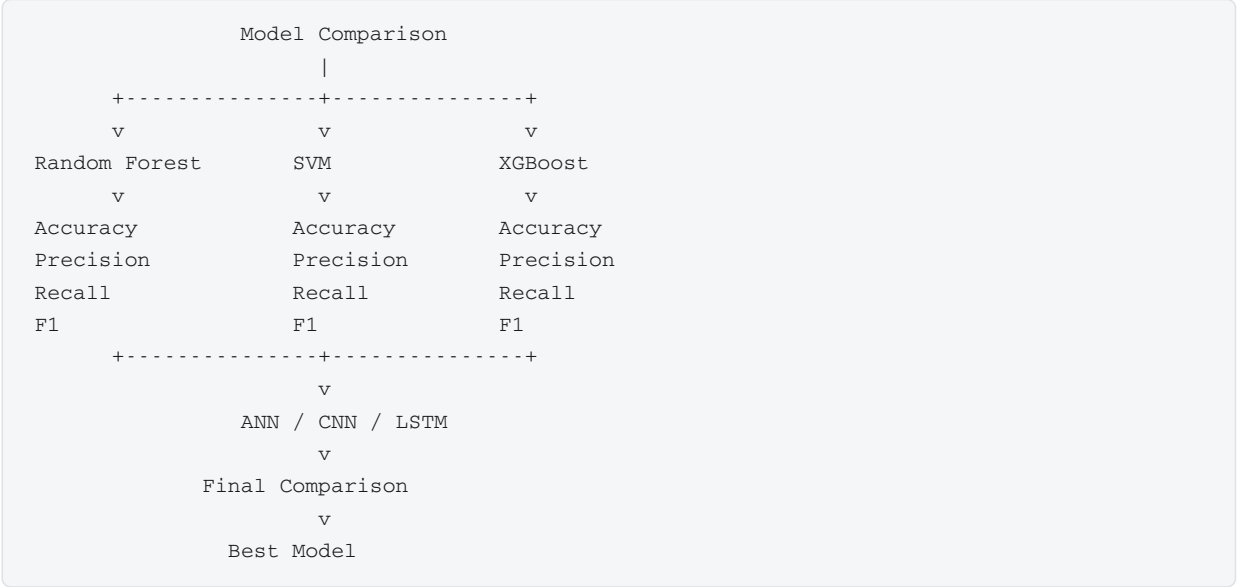
1. Which machine-learning algorithm provides the best intrusion-detection performance on NSL-KDD?
2. Can deep learning improve intrusion detection compared with conventional ML on the same data?
3. Which network features have the greatest influence on attack classification?
4. How does class imbalance affect detection of minority attack classes (R2L, U2R)?
5. Which attack categories are most difficult for the models to classify, and why?
6. Can the best-performing model be integrated into a practical, real-time dashboard?
7. What is the trade-off between detection performance and prediction time?
8. Does a model trained on NSL-KDD generalize to a more modern dataset like CICIDS2017 or UNSW-NB15, or does performance drop significantly (a proxy for real-world deployment readiness)?

---

18. Expected Final Result

```
Input
  |
```

```
Network Connection
  |
Preprocessing
  |
AI Model
  |
Prediction
  |
+---------------------------+
| Normal                    |
| DoS                       |
| Probe                     |
| R2L                       |
| U2R                       |
+---------------------------+
  |
Confidence Score
  |
Dashboard
  |
Alert / Report
```



Important: do not invent your final accuracy numbers before running the experiments. Your report should present only the actual results produced by your implementation, including any results that are lower than expected — an honest discussion of limitations is usually valued more highly by examiners than an inflated success story.

---

19. Ethical, Legal, and Practical Considerations

A short section like this in your report signals maturity to examiners:

- **Use only public, licensed benchmark datasets** (NSL-KDD, CICIDS2017, UNSW-NB15) for training and evaluation — do not capture real traffic from networks you do not own or have permission to monitor.
- **False positives have a real cost.** A dashboard that raises too many false alerts will be

ignored by operators ("alert fatigue"); discuss this trade-off explicitly when choosing your operating threshold.
- **A benchmark accuracy is not a deployment guarantee.** Models trained on NSL-KDD may not generalize to a live network with different traffic patterns; state this clearly as a limitation rather than a criticism-proof claim of "real-world readiness."
- **License compliance.** If you reuse code structure or ideas from any repository listed in Section 15, check its license (MIT, Apache-2.0, GPL, or "no license" i.e. all rights reserved) and cite the author in your report's references/acknowledgments section.

---

20. Suggested Report / Chapter Structure

1. Introduction & Problem Statement
2. Literature Review (cite the IEEE/Elsevier papers referenced by repositories in Section 15.2)
3. Dataset Description (NSL-KDD, and optionally your secondary dataset)
4. Methodology (preprocessing → feature engineering → model training)
5. Experimental Setup (train/test split, hyperparameters, hardware/software used)
6. Results (binary classification, multiclass classification, per-model comparison table)
7. Discussion (feature importance, class-imbalance impact, error analysis on misclassified samples)
8. Dashboard / Deployment Demo
9. Limitations and Future Work
10. Conclusion
11. References

---

21. Final Recommendation

For a final-year project, a strong working title is:

> ***"Design and Implementation of an AI-Based Network Intrusion Detection System Using Machine Learning and Deep Learning"***

Recommended implementation scope:

NSL-KDD → preprocessing → Random Forest + SVM + Decision Tree + XGBoost → ANN/CNN/LSTM → model comparison → feature importance (SHAP) → Streamlit dashboard → intrusion alerts → (optional) generalization test on CICIDS2017 or UNSW-NB15.

This is a realistic, sufficiently advanced scope for a single semester, and the reference repositories in Section 15 give you working code to study at every stage — from preprocessing, to modeling, to dashboarding — without needing to copy any of them directly.